Lesson 7 Teacher's Key: On the Move

Unity - Instructor Reference Only

VIDEO GAME CREATION

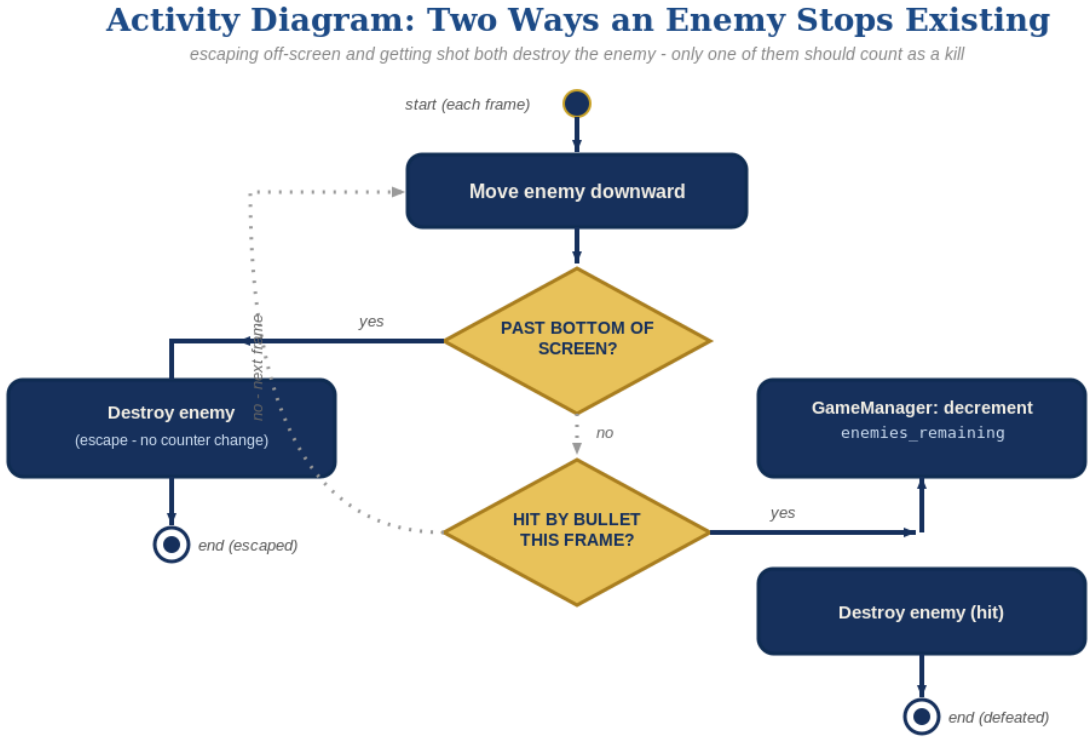
What This Lesson Teaches

The same escape-vs-hit distinction as Godot, expressed via Destroy(gameObject) called directly from the enemy's own Update() vs. GameManager.RegisterKill() called only from the bullet's OnTriggerEnter2D.

The Bug Being Fixed

Unity's Lesson 6 design already called RegisterKill() exclusively from Bullet.cs's OnTriggerEnter2D, so there is no bug to fix here - only new code to add, same as Godot.

Level 1 - Full Worked Solution



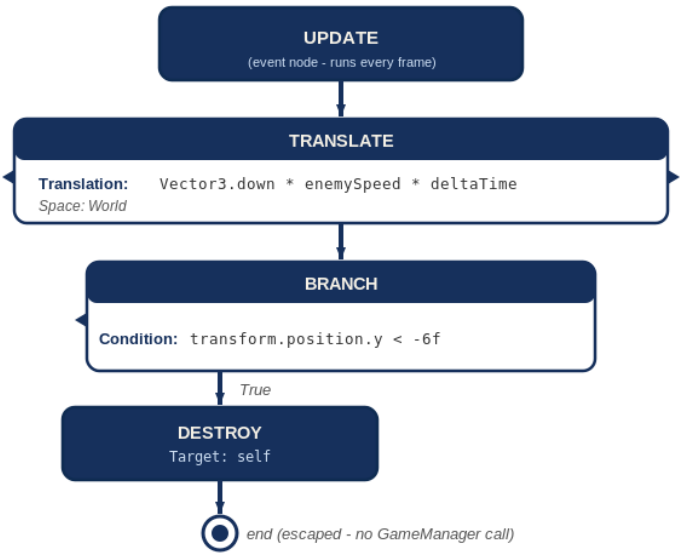
These two checks look similar - both end with "the enemy is gone" - but only the hit check is allowed to touch enemies_remaining. Attaching the counter to a generic "was destroyed" hook instead of this specific hit check is exactly the bug GameMaker and 001 Game Creator have to fix this lesson.

The general escape-vs-hit distinction behind this week's exercise - the same shape in every engine, whatever the exact code or blocks look like.

- transform.Translate(Vector3.down * enemySpeed * Time.deltaTime) is frame-rate-independent movement, consistent with patterns taught since Lesson 2.
- The Y threshold (-6f) assumes the default Lesson 1-3 orthographic camera setup; note in class if a student customized their camera size.
- Destroy(gameObject) in Enemy.cs's own Update() never calls GameManager.RegisterKill() - that call stays exclusively in Bullet.cs.

Unity Visual Scripting: Level 1 Movement and Escape Cleanup

Enemy prefab's Script Graph - the node-for-node equivalent of the Level 1 code



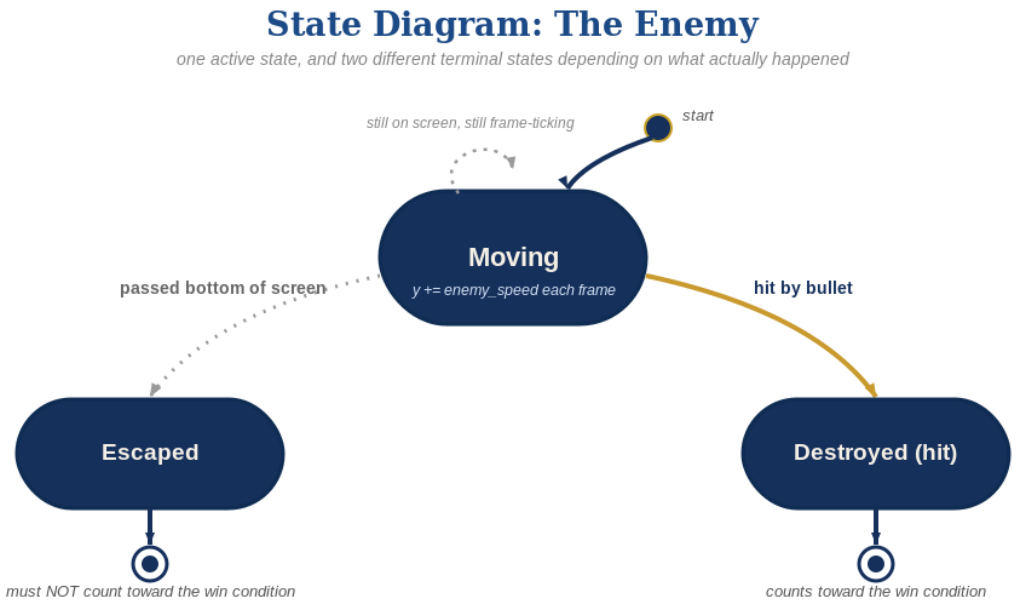
Compare this graph to Bullet.cs's OnTriggerEnter2D in the Key: only that separate event graph ever calls GameManager.RegisterKill() - the Enemy prefab's own Update graph above never references it at all.

GameManager already existed before this lesson - Unity was the one engine already using a dedicated manager object.

The Enemy prefab's Update graph, in Unity Visual Scripting - the node-for-node equivalent of the Level 1 code.

Why This Structure

- Same principle as Godot: RegisterKill() is only ever invoked from the one code path that actually represents a hit (Bullet.cs's trigger callback), and Enemy.cs's own escape-cleanup code has no reason to know that method exists.
- Worth pointing out to students that this is a general engineering habit worth keeping for the rest of the course: when adding a new way for something to end, check whether existing 'on end' code was written assuming there was only one way.



Once an enemy leaves Moving, it never comes back - both destinations are final. The whole trick this lesson teaches is making sure your code can tell these two final states apart, since only one of them is a kill.

The enemy's two possible endings this week - only one of them should count as a kill.

Level 2 / Level 3 Talking Points

- Level 3 structure: same talking point as Godot - ask students to explain in their own words why Enemy.cs shouldn't call RegisterKill() even though it 'also' destroys the enemy.
- For strong students: ask what would happen if Bullet.cs's OnTriggerEnter2D destroyed the enemy but Update() also destroyed it the same frame (a double-destroy race) - a good lead-in to why Level 3 error-hardening exists later in the course.

Common Student Mistakes

- Calling GameManager.RegisterKill() from Enemy.cs's Update() 'just in case' - same bug pattern as Godot, explicitly avoid.
- Using Time.deltaTime incorrectly (multiplying twice, or forgetting it) resulting in wildly wrong speeds on different frame rates.
- Not adjusting the -6f threshold for a customized camera - enemies either never despawn or despawn while still visible.